T08 Damage, Knockback and Death

CRE132 worksheet

What you'll make

Fights that feel like fights. Hit a slime with the sword: it's pushed back, flashes, and can't be hit again for a moment. The third hit bursts it into bubbles, and more often than not it leaves a coin or a potion behind. When a slime jumps on you, you're knocked back and flash for a second, during which nothing can hurt you. Lose all 5 hearts and the player collapses and the Game Over panel appears.

By the end you'll know how to do **knockback** in any direction, what **invincibility frames** ("i-frames") are for, and how Health's **On Died** event can set off effects and drops without Health knowing about any of them.

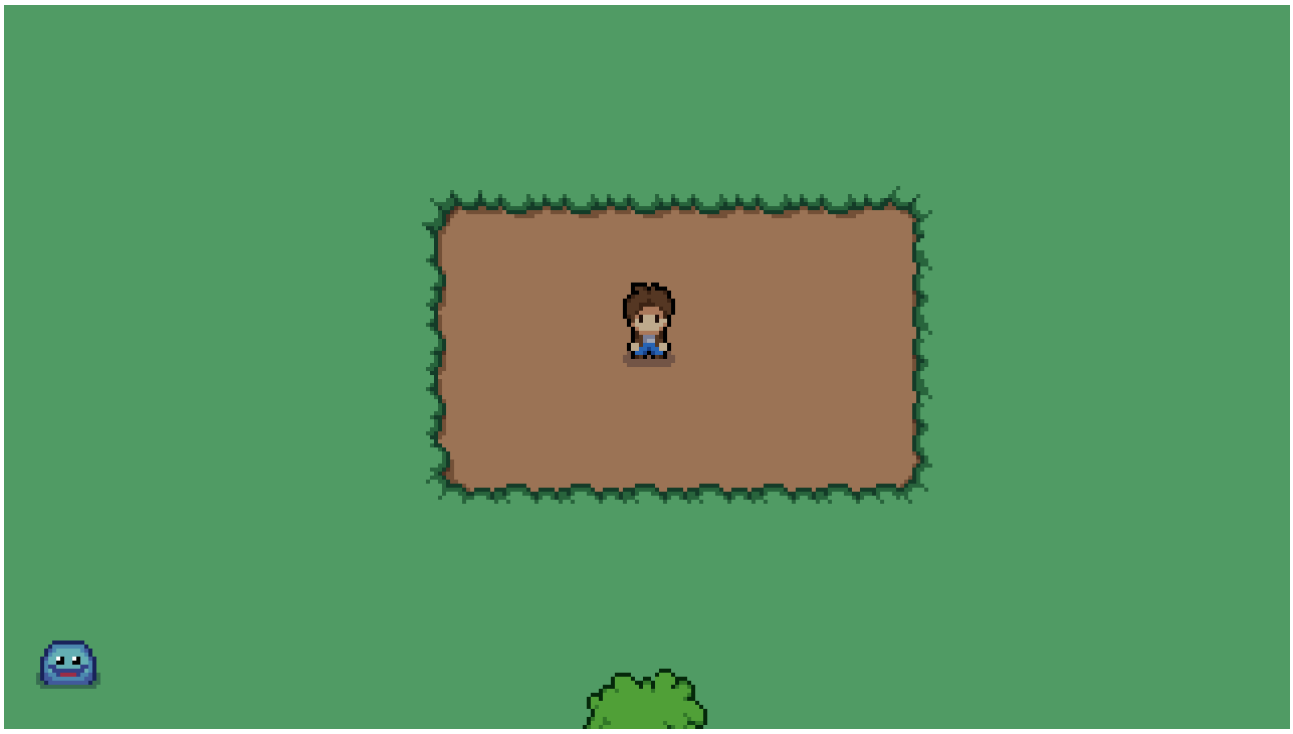


Figure 1: The finished meadow: the player on a patch of dirt, with a slime in the corner and a bush below.

Before you start

1. Unzip `CRE132_T08_DamageKnockbackDeath_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_T08_DamageKnockbackDeath_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/TopDown/T08_DamageKnockbackDeath`.
3. Press **Play**. The four slimes behave as in T06 and you can fight them with the sword (J), but hits just turn things redder: no pushing, no flashing, and things simply vanish at zero health. Press Play again to stop.

Under **Characters** are the **Player** (**Health** 5) and **Slime 1** to **Slime 4**. The scene also has S06's managers (**GameManager**, **AudioManager**, **Scene Loader**) and the **Game Over Canvas**. The

prefabs you'll need are in `Assets/_game/Prefabs/Effects` (**Slime Death**, **Player Death**) and `Assets/_game/Prefabs/Collectables` (**Health Potion**, **Coin**).

> **Making a script:** in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. Getting hurt: TopDownHurt

Make a folder `Assets/_game/Scripts/T08_DamageKnockbackDeath` and create `TopDownHurt` in it:

```
// How a top-down character reacts to a hit: lose health, get pushed away from the hit,
// then flash for a short time while nothing can hurt it again (invincibility frames, or
// "i-frames"). The top-down version of P09's PlayerHurt: knockback works in any
// direction, and it can go on enemies too. The sword (T04) and slime attacks (T06) call
// Hurt when it's there.
using System.Collections;
using UnityEngine;

public class TopDownHurt : MonoBehaviour
{
    [SerializeField] private float knockbackSpeed = 8f;
    [SerializeField] private float knockbackTime = 0.15f;
    [SerializeField] private float invincibleTime = 1f;
    [SerializeField] private float flashInterval = 0.1f;

    // The script that moves this character (TopDownMovement on the player, EnemyAI on a
    // slime). It's switched off during the knockback, or it would set its own velocity
    // straight away.
    [SerializeField] private MonoBehaviour movementToPause;

    private Health health;
    private Rigidbody2D body;
    private SpriteRenderer spriteRenderer;
    private bool isInvincible;

    private void Awake()
    {
        health = GetComponent<Health>();
        body = GetComponent<Rigidbody2D>();
        // The sprite is on a child (see T03).
        spriteRenderer = GetComponentInChildren<SpriteRenderer>();
    }

    // Called by anything that hurts this character, with the position the hit came from.
    public void Hurt(int damage, Vector2 hitFrom)
    {
        if (isInvincible)
        {
            return;
        }

        health.TakeDamage(damage);

        // That hit was fatal: Health takes it from here.
        if (health.CurrentHealth <= 0)
        {
            return;
        }
```

```
        StartCoroutine(Knockback(hitFrom));
        StartCoroutine(Invincibility());
    }

    private IEnumerator Knockback(Vector2 hitFrom)
    {
        // Away from the hit: the arrow from the hit to us, made length 1.
        Vector2 awayFromHit = ((Vector2)transform.position - hitFrom).normalized;

        // Switch the movement off first: TopDownMovement stops the body when it's
        // switched off.
        movementToPause.enabled = false;
        body.linearVelocity = awayFromHit * knockbackSpeed;

        yield return new WaitForSeconds(knockbackTime);

        body.linearVelocity = Vector2.zero;
        movementToPause.enabled = true;
    }

    private IEnumerator Invincibility()
    {
        isInvincible = true;

        // Flash: hide and show the sprite every flashInterval seconds.
        float timer = 0f;
        while (timer < invincibleTime)
        {
            spriteRenderer.enabled = !spriteRenderer.enabled;
            yield return new WaitForSeconds(flashInterval);
            timer = timer + flashInterval;
        }

        spriteRenderer.enabled = true;
        isInvincible = false;
    }
}
```

What's going on:

- **`Hurt` is what attackers call** instead of `TakeDamage`. It passes on the damage, then starts two coroutines that run side by side: the knockback and the invincibility.
- **`isInvincible` blocks every hit until the flashing ends.** Without it, a slime touching you for several frames could take all your health at once, and you'd have no moment to escape.
- **Knockback in any direction:** `myPosition - hitFrom` is the arrow from the hit to us; normalised, it points straight away from the hit. P09's platformer version only pushed left or right.
- **Switch the movement off first.** `movementToPause` is whichever script moves this character. If it stayed on, it would set its own velocity at the next physics step and cancel the push.
- **`MonoBehaviour` as the slot's type** means any script fits: **TopDownMovement** on the player, **EnemyAI** on a slime.
- **The flash** turns the Sprite Renderer off and on every 0.1 seconds, then makes sure it ends switched on.

Add it to the **Player**: leave **Knockback Speed** 8, **Knockback Time** 0.15, **Invincible Time** 1 and **Flash Interval** 0.1. Drag the Player's own **Top Down Movement** component (click and hold its title in the Inspector) into the **Movement To Pause** slot.

Then add it to **each slime** in turn: set **Invincible Time** to **0.3** and drag that slime's own **Enemy AI** component into **Movement To Pause**.

Nothing calls `Hurt` yet, so nothing changes in Play. The next two parts fix that.

2. The sword uses it: changes to AttackHitbox

Open T04's `AttackHitbox` (in `Assets/_game/Scripts/T04_MeleeAttack`). Below the `alreadyHit` line, add:

```
    // The character swinging the sword: the nearest Rigidbody2D above this object (the
    // player's root). Not transform.root, which is the top of the Hierarchy (a
    // "Characters" folder object, for example).
    private Transform owner;

    private void Awake()
    {
        owner = GetComponentInParent<Rigidbody2D>().transform;
    }
```

In `OnTriggerEnter2D`, change the "don't hit ourselves" check from `transform.root` to `owner`:

```
        // Don't hit ourselves.
        if (health.transform == owner)
        {
            return;
        }
```

At the end of `OnTriggerEnter2D`, replace `health.TakeDamage(damage);` with:

```
        // T08: if it can be knocked back, hurt it through TopDownHurt, which pushes it
        // away from the attacker and gives it a moment of invincibility. Otherwise it
        // just takes the damage.
        TopDownHurt hurt = health.GetComponent<TopDownHurt>();
        if (hurt != null)
        {
            hurt.Hurt(damage, owner.position);
        }
        else
        {
            health.TakeDamage(damage);
        }
```

- **`owner` is the character swinging the sword**: the nearest Rigidbody2D above the hitbox, which is the Player root. The knockback pushes enemies away from its position.
- **Why not `transform.root`?** It's the top of the Hierarchy, not "my character". The Player sits inside a **Characters** object, so the root is that object, at (0, 0). Ask for the component you mean instead.
- **Anything without TopDownHurt still just takes damage**, so the T04 dummies would work as before.

Test it: press Play and hit a slime. It's pushed away from you and flashes briefly.

3. The slime's attack uses it: a change to EnemyAI

Open T06's `EnemyAI`. In `UpdateAttack`, replace `player.GetComponent<Health>().TakeDamage(attackDamage);` with:

```
                // T08: through TopDownHurt if the player has it (knockback and
                // invincibility).
```

```
                TopDownHurt hurt = player.GetComponent<TopDownHurt>();
                if (hurt != null)
                {
                    hurt.Hurt(attackDamage, transform.position);
                }
                else
                {
                    player.GetComponent<Health>().TakeDamage(attackDamage);
                }
```

The slime passes its own position, so you're pushed away from it.

Test it: let a slime jump on you. You're knocked back and flash for a second; another slime can't hurt you while you flash.

4. Leaving something behind: DeathEffect

Create `DeathEffect` in the `T08_DamageKnockbackDeath` folder:

```
// What's left behind when something dies: a death animation, and sometimes a dropped
// item. Health's On Died event calls Spawn(), just before Health destroys the object.
using UnityEngine;

public class DeathEffect : MonoBehaviour
{
    // A prefab that plays a death animation once.
    [SerializeField] private GameObject effectPrefab;
    // How long the effect stays before it's removed (the length of its animation).
    [SerializeField] private float effectTime = 0.6f;

    // Things it might drop. One is picked at random.
    [SerializeField] private GameObject[] dropPrefabs;
    // 0 = never drops anything, 1 = always drops something.
    [SerializeField] [Range(0f, 1f)] private float dropChance = 0.5f;

    public void Spawn()
    {
        if (effectPrefab != null)
        {
            GameObject effect = Instantiate(effectPrefab, transform.position,
                Quaternion.identity);
            Destroy(effect, effectTime);
        }

        // Random.value is a random number from 0 to 1, so it's below 0.5 half of the
        // time.
        if (dropPrefabs.Length > 0 && Random.value < dropChance)
        {
            // Random.Range with whole numbers never returns the top number, so this is 0
            // to Length - 1.
            int index = Random.Range(0, dropPrefabs.Length);
            Instantiate(dropPrefabs[index], transform.position, Quaternion.identity);
        }
    }
}
```

- **`Spawn` is public** so Health's **On Died** event can call it, just before Health destroys the object.
- **`Destroy(effect, effectTime)`** removes the effect after that many seconds: the length of its animation.

- **`Random.value`** is a random number from 0 to 1. With **Drop Chance** 0.6, it's below 0.6 six times out of ten, so slimes drop something more often than not.
- **`Random.Range(0, dropPrefabs.Length)`** picks a random index. With whole numbers it never returns the top number, so with two prefabs you get 0 or 1, never 2.
- **`[Range(0f, 1f)]`** shows the drop chance as a slider in the Inspector.

Add it to **each slime**:

1. Add **Death Effect**. **Effect Prefab**: **Slime Death**. **Effect Time**: **0.5**. **Drop Prefabs**: click **+** twice and drag in **Health Potion** and **Coin**. **Drop Chance**: **0.6**.
2. On the slime's **Health**, find **On Died ()**. Click **+** to add a row. Drag the slime itself (from the Hierarchy) into the row's object slot, then choose **DeathEffect > Spawn ()** from the function menu.

Then the **Player**: add **Death Effect** with **Effect Prefab Player Death**, **Effect Time 0.6**, no drop prefabs and **Drop Chance 0**. Connect its Health's **On Died ()** to **DeathEffect > Spawn ()** the same way.

Do the slimes one at a time: each event and each slot must point at that slime's own components.

Test it: defeat a slime. It bursts into bubbles and often leaves a coin or a potion.

5. Potions are for the player: a change to HealthPickup

A slime walking over a dropped potion would drink it, because HealthPickup heals anything with Health. Open S01's `HealthPickup` (in `Assets/_game/Scripts/Health`). Below the `healAmount` line, add:

```
    // Tick for potions dropped among enemies (T08), so a slime walking over one doesn't
    // drink it.
    [SerializeField] private bool playerOnly = false;
```

and at the very start of `OnTriggerEnter2D`, before the `GetComponent` line, add:

```
        if (playerOnly && !other.CompareTag("Player"))
        {
            return;
        }
```

- **`playerOnly`** **is** **`false`** **by default**, so the potions in earlier samples still heal anything.
- **`CompareTag("Player")`** checks the tag, like `FindWithTag` in T06.

Add it: open the **Health Potion** prefab and make sure **Player Only** is ticked on its **Health Pickup** (the prefab may already have it ticked now that the field exists). Leave **Heal Amount** at **2**.

6. Game over

There's nothing to add here: it all fits together already. When your health reaches zero, Health calls **On Died** (the collapse animation from **Player Death**) and destroys the Player. S06's **GameOver-Screen** on the **Game Over Canvas** notices that the player has gone and shows the panel, with **Try Again** and **Main Menu**.

Test it: let the slimes beat you. The player collapses, then the Game Over panel appears. **Try Again** reloads the scene.

Check your work

- ☐ Hitting a slime pushes it away from you and it flashes; a third hit bursts it into bubbles.
- ☐ Defeated slimes often leave a coin or a potion, and slimes don't drink the potions.

- ☐ Coins add to the score; potions heal you by 2.
- ☐ A slime's jump knocks you back, and you flash for a second without taking more damage.
- ☐ At zero health the player collapses and the Game Over panel appears.
- ☐ Compare with `CRE132_T08_DamageKnockbackDeath_Finished` if something behaves differently.

Key points

- **Knockback is "stop the normal movement, push, then give it back".**
- **Invincibility frames** stop one hit from counting many times, and give the player a moment to escape.
- **Unity Events** (S09) let death set off effects, drops and game over without Health knowing about any of them.
- **`transform.root` is the top of the Hierarchy, not "my character".** Ask for the component you mean.

If something goes wrong

- **NullReferenceException in TopDownHurt's Knockback:** the **Movement To Pause** slot is empty.
- **No knockback, but the hit counts:** the movement script in **Movement To Pause** belongs to a different object, so the right one was never switched off. Drag in the component from the same object.
- **The bubbles or the drop appear where a different slime is:** that slime's **On Died** event points at another slime's DeathEffect. Check each slime's own components.
- **Nothing happens when a slime dies:** its Health's **On Died ()** has no **DeathEffect > Spawn ()** entry, or the row's object slot still says **None (Object)**.
- **NullReferenceException in AttackHitbox, on `owner`:** the `Awake` method you added is missing or misspelt (`Awake`, capital A), so `owner` was never set.
- **Slimes drink the potions:** make sure **Player Only** is ticked on the **Health Potion** prefab itself.

Try it

1. Knock the player back further for bigger hits (multiply by `damage`).
2. Add a third drop with a lower chance: a gem worth 5.
3. Show a heart bar (S03's **HealthBar**) instead of relying on the red tint.
4. Wait a second after the player dies before showing Game Over.